How Is Behavioral Equivalence Evaluated in LLM-Based Code Translation and Migration? A Systematic Mapping Study

Author Name 1
Institute 1
City, Country
author1@example.com

Author Name 2
Institute 2
City, Country
author2@example.com

Author Name 3
Institute 3
City, Country
author3@example.com

Author Name 4
Institute 4
City, Country
author4@example.com

Author Name 5
Institute 5
City, Country
author5@example.com

Abstract

Large language models (LLMs) are now used to translate code between languages and to migrate it across versions, libraries and frameworks. Whether the migrated code behaves like the original is always decided by some oracle, and the choice of oracle can change the reported result. We present a systematic mapping study of how behavioral equivalence is evaluated in this line of research, covering the main proceedings of ICSE, FSE and ASE from 2024 to 2026. From 4,451 papers we screened 431 candidates and included 51 (49 in the main corpus, 2 on cross-language equivalence detection), each read in full. Three findings stand out. First, compilation (67%) and existing tests (57%) dominate, while formal verification (4%) and LLM judges (6%) are rare; 9 papers use no execution-based oracle at all. Second, the oracle follows the context: no legacy migration uses existing tests, library migration relies on human judgment (9 of 12 papers), and LLM-generated tests and LLM judges appear only in 2026. **[Hà Cao:** Three RQ2 claims here need hedging (see Section 5.3): only 3 of the 12 LIB papers rely on people alone (8 of 12 also run code), so "relies on" overstates it; "no legacy migration uses existing tests" depends on our narrow legacy definition; LLM-generated tests are new only as final-output oracles. LLM judges become 4% if ReCodeAgent's O-h is recoded.**]** Third, 33 papers report two or more oracles on the same outputs; among the 18 that report both a compile rate and a behavioral pass rate, compile rates exceed behavioral pass rates by up to 98 percentage points. Nine papers state no limitation of their own oracle. We contribute a coded map of oracle practice, a catalog of how oracles disagree on the same outputs, a classification of the limitations studies report about their own oracles, six gaps between evaluation practice and real migration, and a checklist for reporting oracle provenance.

Keywords

code translation, code migration, large language models, test oracle, behavioral equivalence, systematic mapping study

1 Introduction

Code migration is routine industrial work, and LLMs are taking a growing share of it. Google reports LLM-driven migrations across its monorepo, such as JUnit 3 to JUnit 4 [40], and similar efforts span C-to-Rust ports, COBOL modernization and dependency upgrades. In every case someone must decide whether the migrated code still does what the original did. That decision rests on an *oracle*: a compiler, a test suite, a comparison with the original program, a formal check, a human or another LLM.

The oracle is not a neutral instrument. Papers in our own scope show the same outputs looking very different under different oracles:

- DepWareTrans [9]: file-by-file Java-to-Kotlin migration reaches 38.16% compilation but only 9.39% test success.
- DGIR [15]: a model rewarded only for compiling reaches 97.9% compilation and 0% computational accuracy.
- DepUpgrade [35]: in 96.75% of dependency-upgrade tasks the code compiles, but reviewers judge only 62 of the 123 tasks (50.4%) to end with a semantically equivalent patch.

Work at co-located events reports the same pattern: Copilot Agent Mode migrating SQLAlchemy reaches a median migration coverage of 100% but a median test pass rate of 39.75% [2]. The problem is not specific to translation either: in an FSE 2025 study, existing regression tests caught only 7.6% of behavior-changing commits [20].

Despite this, no study has mapped which oracles the community uses, where those oracles come from, and in which settings. The closest related studies (Section 3) survey translation techniques, benchmarks, publication trends or security, but none analyzes where equivalence oracles come from or how reliable they are. We address that gap with a systematic mapping study of three flagship SE venues, and ask:

RQ1. Which oracles are used to evaluate behavioral equivalence in LLM-based code translation and migration?

RQ2. Where do these oracles come from, and in which contexts are they used: with or without existing tests, for which kind of migration, in which track? **[Hà Cao:** No field codes whether the studied systems have tests (Table 4); Section 5.3 uses the legacy flag as an unstated proxy. Either add a test-availability field (yes / partial / no / unstated) or rephrase RQ2 as legacy vs. non-legacy and say so. RQ2 also reports years, which the RQ does not mention.**]**

RQ3. Which threats to oracle reliability do the studies acknowledge or demonstrate?

RQ4. What gaps separate current evaluation practice from the needs of real migration projects, and what should studies report and investigate next?

Our contributions are: (1) a coded map of 49 primary studies by oracle, oracle origin, migration kind, track and LLM role, with the

empty cells that mark under-studied combinations; **[Hà Cao:** LLM role is never crossed with oracles in Section 5. I can add LLM-role rows to Table 6: all 5 O-e and all 3 O-h papers propose a technique; none is an empirical study or a benchmark. Otherwise drop "LLM role" here. Most empty cells are also expected by chance (see Section 5.3).**]** (2) a catalog of the 33 studies that report two or more oracles on the same outputs, showing how far those oracles disagree; (3) a classification of the limitations that studies report about their own oracles; and (4) six gaps between evaluation practice and real migration, with a checklist for reporting oracle provenance. All data and coding are released (Section 8).

Structure. Section 2 defines the kinds of migration and the oracle categories we code. Section 3 positions this study against the closest related studies. Section 4 reports the search, selection and extraction. Section 5 answers RQ1 to RQ3, and Section 6 answers RQ4 and proposes the checklist. Section 7 discusses threats to validity, and Section 8 concludes. Appendix A lists every included paper with its coding.

2 Background

Kinds of migration. We distinguish five kinds, and a study may involve several: between programming languages (XL, e.g. C to Rust, COBOL to Java); between versions of a language or platform (VER, e.g. Java 8 to 17); between libraries or APIs, including dependency upgrades with breaking changes (LIB, e.g. JUnit 3 to 4); between dialects (DIA, e.g. PostgreSQL to MySQL); and between frameworks or application platforms (FW, e.g. Android to iOS, PyTorch to Paddle). A *legacy* flag marks a legacy source system (COBOL, PL/I, ASP.NET WebForms). **[Hà Cao:** Define "legacy" operationally. EvoC2Rust, SmartC2Rust and Code hints call their sources legacy but are not flagged. With them, 2 of 8 legacy papers use existing tests (SmartC2Rust, EvoC2Rust), and "no legacy paper uses existing tests" (Section 5.3) no longer holds. Justify the narrow definition or report both.**]**

Behavioral equivalence. Two programs are behaviorally equivalent if, for every input, they produce the same observable behavior: the same outputs, the same effects on files, databases or other state, and the same termination or failure. Checking this exactly is undecidable in general, so every practical check approximates it. Tests and differential runs sample the input space; bounded model checking and symbolic execution cover it up to a bound; formal proofs cover it completely but only for a model of the program; and compilation, similarity metrics and human or LLM judgment do not run the program at all. Migration also relaxes the definition in ways that a check must state. A translation may map types whose ranges differ, such as 32-bit integers in C into Python's unbounded integers; a library migration may keep the functional result but change logging, performance or error messages; and a framework migration may be judged on what a user sees rather than on internal state. "Equivalent" therefore always means equivalent with respect to some oracle and some notion of observable behavior.

Oracles. An oracle decides whether the migrated program behaves like the original. We use nine categories (Table 1).

The categories are not exclusive. Many studies combine several, and some use one inside the technique (for example, to filter training data or drive a repair loop) and another to report results. Section 4.5 states which of these uses we code. **[Hà Cao:** O-e and O-f overlap. When LLM-written tests take their expected outputs from running the original, TRAVEL is coded O-e, PROGnosticator O-f and PtrTrans both; BatFix is coded O-c although the original program's output is its oracle. Table 1 mixes two axes: the kind of check and the source of the expected output. Proposed rule: expected outputs from running the original = O-f, whoever wrote the inputs; the input source goes to the origin field.**]**

3 Related Work

Closest related studies. Chen et al. present the most recent systematic literature review (SLR) on neural code translation, analyzing 57 primary studies from seven perspectives, including evaluation subjects and metrics [10]. Their catalog records which metrics appear; we take evaluation itself as the object of analysis, classifying where oracles come from and how reliable they are, and contrasting settings with and without tests. Jiao et al. propose an evaluation taxonomy and benchmark for neural code translation [27]; their taxonomy classifies translation task types in the pre-LLM era, whereas we map evaluation practice in the LLM and agent era, including LLM-generated tests and LLM judges. A security-focused systematization of knowledge (SoK) of AI4Code sketches verification techniques for translation within one subsection [53]; we give the topic a dedicated, corpus-based treatment. A recent systematic mapping study of generative AI for code translation maps the publication landscape by year, venue, research type, dataset and model [48]; it does not analyze oracle origin or reliability, which are our central axes.

The oracle problem. Barr et al. survey the test oracle problem in general [6]. Migration is a special case in which the original program is available as a reference, so differential and equivalence-based oracles become possible; our map shows how rarely that advantage is used (13 of 49 papers; Figure 2) **[Hà Cao:** 13 counts O-f only. Papers that use the original program as the reference are 14 with TRAVEL (O-e, e:R) and 15 with BatFix. Either say "as a differential oracle (O-f)" or report the broader count; the number also changes if O-e/O-f is recoded.**]**.

Individual studies. Individual benchmarks and techniques already work around the oracle problem: coverage gates against reward hacking [37], tests translated with the code [24], differential fuzzing without predefined tests [12], and benchmarks for equivalence judgments [50]. These works document the defenses and failure modes that we consolidate. Two of them, FreshBrew and AlphaTrans, are in our corpus. Of the other two, one studies refactoring within a single language and fails our criteria, and the other appeared at ESEM, outside our venues.

4 Method

We followed the guidelines for systematic mapping studies in software engineering [45]. The protocol, search log, codebook and all decisions are in the replication package.

Table 1: Oracle categories used throughout the paper.

Code	Oracle	What it checks	Known weakness
O-a	Surface similarity (BLEU, CodeBLEU, exact match, AST match)	Closeness to a reference translation	Equivalent code can look different, and similar code can behave differently
O-b	Compilation, type checking or another well-formedness check	The output is well-formed in the target	Says nothing about behavior
O-c	Existing tests	Behavior on the project's or benchmark's tests	Only as strong as the tests; benchmarks often have few
O-d	Translated tests	Behavior on tests migrated alongside the code	Test and code may share the same translation error
O-e	LLM-generated tests	Behavior on new tests written by an LLM	Generated expectations can be wrong or aligned with the output
O-f	Differential testing against the original	Same outputs as the source program on the same inputs	Limited by input generation; needs the source to run
O-g	Formal verification	Equivalence proven or bounded-checked against the original or a specification	Costly; bounds and models limit coverage
O-h	LLM judge	An LLM's verdict on equivalence	Reliability of the judge is itself unvalidated
O-i	Human evaluation	A person's judgment	Subjective, small samples, rarely agreement-measured

4.1 Scope

A paper is in scope if it appears at ICSE, FSE or ASE in 2024, 2025 or 2026 in any track (research, industry/SEIP, NIER, Ideas, Visions and Reflections, demonstrations, posters and journal-first), whether it is published in the conference's main or Companion volume or, for journal-first papers, presented there. We call this set the main proceedings. These venues are where SE evaluation norms are set and accepted by reviewers, and their industry tracks are where legacy settings without tests appear. Co-located workshops and conferences (ReCode, LLM4Code, FORGE, AGENT, AIware, AISM, ICPC, MSR and others) are out of scope: the set of events changes from year to year, so counting them would make years incomparable. ISSTA 2025 and Internetware 2025, which were co-hosted with FSE that year only, are excluded for the same reason. Related papers outside the corpus are cited in Sections 1, 3 and 6 but not counted.

4.2 Search

We applied one frozen keyword string to titles and abstracts:

```
translat* OR transpil* OR migrat* OR moderni* OR legacy
OR porting OR ported OR upgrad* OR deprecat* OR
"cross-language" OR "cross-lingual" OR equivalen*
```

The string omits "oracle" and "behavior*", because they pulled in unrelated testing papers, and omits "LLM", because many neural papers do not use the word; the LLM criterion is applied at screening. Paper lists came from each conference program on conf.researchr.org, covering every track that had a published paper list (three poster tracks did not; Section 7). Both the main and the Companion volume of each conference were cross-checked through Crossref, and missing abstracts were taken from Semantic Scholar and arXiv. The final scan, on October 23, 2026, covered 4,451 main-proceedings papers and returned 430 keyword hits; for 80 papers without an available abstract, the string was matched against the title only. Table 2 gives the counts for each conference and year. Most papers without an abstract are from ICSE 2024 (61 of 80).

Table 2: [new] Search and selection by conference and year. Papers: main-proceedings papers scanned; No abstract: papers matched on their title only; Hits: keyword hits; Candidates: hits plus the one known paper the search missed; Included: main corpus; I3: cross-language equivalence detection tier.

Venue	Papers	No abstract	Hits	Candidates	Included	I3
ICSE 2024	611	61	58	58	2	0
ICSE 2025	526	0	41	41	4	0
ICSE 2026	796	1	68	68	7	0
FSE 2024	261	6	17	17	2	0
FSE 2025	451	1	35	36	6	1
FSE 2026	563	2	64	64	11	1
ASE 2024	299	1	25	25	4	0
ASE 2025	447	1	53	53	4	0
ASE 2026	497	7	69	69	9	0
Total	4,451	80	430	431	49	2

To estimate recall, we used 40 in-scope papers that meet our criteria and that we had identified before the scan, in exploratory searches of the same proceedings and while reading related work. The string retrieved 39 of the 40 (97.5%). We added the one miss, a dependency-repair paper whose title and abstract use none of our terms [16], to the candidates as a known paper, which gives 431 candidates.

4.3 Selection

A paper is included if it satisfies I4 and at least one of I1–I3, and no exclusion criterion (Table 3).

Nine clarification rules settle recurring hard cases. Migrating tests between languages or libraries counts as I1 (L-TEST). A deterministic technique counts if it compares against an LLM baseline using the same oracle and reports numbers (L-BASE). Microservice extraction, architecture changes and decompilation do not count,

Table 3: Inclusion (I) and exclusion (E) criteria.

Code	Criterion
I1	Uses a neural model or LLM to translate or migrate code between a named source and target (language, version, dialect, library or API, framework) and describes how equivalence is checked, even if only by BLEU
I2	Proposes or uses a method, benchmark or oracle to evaluate translations or migrations
I3	Detects equivalence between programs in two different languages; reported as a separate tier
I4	English, peer-reviewed
E1	Natural-language translation, or NL-to-code tasks with no code-to-code translation
E2	No information at all on how outputs are evaluated
E3	Clone or similarity detection that neither targets functional equivalence nor relates to translation
E4	Secondary study
E5	Duplicate of the same work (we keep the most complete version)
E6	Full text unavailable

because there is no source-to-target program pair to compare (L-ARCH, L-DECOMP). A dataset without an evaluation protocol is E2; one with a protocol meets I2 (L-DATA). An oracle or benchmark for rule-based translators still meets I2 (L-RB). Cross-language equivalence detection meets I3 (L-XEQ). Translation must be a main goal or research question of the paper: general models or benchmarks in which translation is one of three or more tasks, and papers that use translation only as a means (to build a dataset, to test a database system), are excluded (L-TASK). A paper that summarizes its authors' published results counts as a duplicate (L-SUM).

Of 431 candidates in the main proceedings, 51 were included and 380 excluded. The reasons were: neither translation nor its evaluation (174); related topics that meet none of I1–I3, such as purely rule-based translation, refactoring with no target, or program repair (96); natural-language tasks (E1, 47); same-language equivalence detection (15); translation as a minor task (L-TASK, 16); no evaluation information (E2, 14); clone detection unrelated to equivalence (E3, 8); duplicates (E5, 7); and no available full text (E6, 3). Hard cases were discussed within the team until agreement was reached.

Figure 1 shows the flow. We excluded 362 of the 431 candidates on title and abstract, 3 of them on the title alone because no abstract was available; Section 4.4 reports the full-text stage.

4.4 Full-text confirmation

Every included paper was read in full (Figure 1). Full texts were sought for 69 candidates. Forty-one of them had first been included on title and abstract: we obtained 29 through freely available versions (arXiv, author pages, PubMed Central, institutional repositories) and 9 through the ACM Digital Library, and excluded 3 IEEE papers with no accessible version under E6. The other 28 candidates were decided on full text from the start, 18 included and 10 excluded. Full-text reading changed 5 of the 38 earlier decisions: one paper explicitly does not validate its translations (E2), one adds type annotations without translating (not I1), one compares two versions

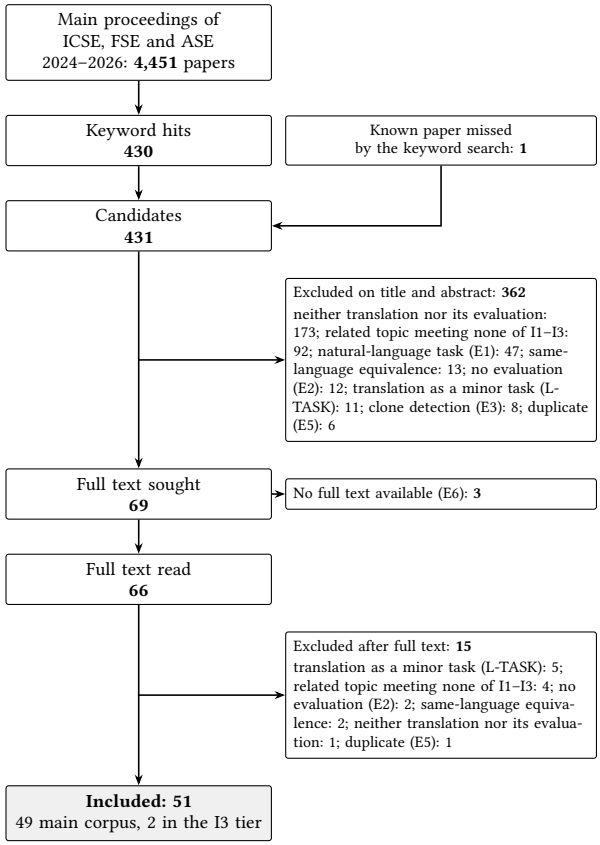


Figure 1: [new] Selection flow. Counts are papers; reasons follow Table 3 and the clarification rules.

of the same PLC program (same-language equivalence), one poster duplicates its full paper (E5), and one deterministic converter fails L-BASE because its LLM baselines are judged by a different oracle. In that last case the authors' LLM-based poster of the same work is counted instead. In total, 51 papers were included (33 confirmed and 18 decided on full text) and 15 were excluded after reading (5 changed decisions and 10 decided on full text).

4.5 Classification and extraction

For each included paper we extracted seven fields: (1) migration kind and legacy flag; (2) track; (3) role of the LLM (proposed technique, evaluated subject, both, baseline only, part of the evaluation tooling, or no LLM); (4) oracles used; (5) the origin of each oracle (existing, translated, LLM-generated, from running the original, from a specification, none); (6) whether two or more oracles were applied to the same outputs, and if so, the numbers under each; and (7) oracle limitations the paper states, quoted with location. We code only oracles that judge the final output; an oracle used solely inside the technique, such as a compiler in a repair loop or a filter on training data, is not counted. All 51 extractions were drafted from the full text, and every number in the disagreement analysis (Section 5.4) was re-checked against the paper. **[Hà Cao:** Boundary cases are coded inconsistently, so the codebook needs

Table 4: [new] Extraction form.

Field	Values
1. Migration kind	XL, VER, LIB, DIA, FW (several allowed); legacy flag
2. Track	Research, industry/SEIP, journal-first, NIER, Ideas, Visions and Reflections, poster, demonstration
3. Role of the LLM	Proposed technique; evaluated subject; both; baseline only; part of the evaluation tooling; no LLM (a smaller neural model, or no model)
4. Oracles	O-a to O-i (Table 1); final-output oracles only
5. Origin of each oracle	Existing (benchmark or project), translated (by the system or by hand), LLM-generated, from running the original, from a specification, none
6. Oracles on the same outputs	Yes or no; if yes, the numbers under each oracle
7. Stated limitations	Free text with location, later grouped into themes (Table 9)

explicit rules before recoding. Examples: a Python syntax check is O-b in AlphaTrans and Python library migration, but parse-only checking is not O-b in Rule synthesis; MBTA's 91.11% compile rate is dropped as "secondary"; IntentTester's migrated tests are the output under evaluation yet are coded O-d; ReCodeAgent's O-h and O-a judge its translated tests, not the translation. A second coder on these boundary papers would also fill the calibration TODO in Section 7.]

Table 4 lists the fields and their values. For field 5 we further coded where the ground truth of each execution-based oracle comes from: for existing tests, whether they were built for a benchmark or belong to a real project's own test suite; for translated tests, whether the system under evaluation translated them or the authors did by hand; for LLM-generated tests, where the expected outputs come from; and for differential testing, where the inputs come from. Tests that benchmark builders translated by hand when building a benchmark count as benchmark tests (O-c); tests that a paper's own authors translate for its evaluation count as translated tests (O-d). Section 5.3 reports the result, and Appendix A gives the code for every paper. **[Hà Cao:** Consider two more fields: (a) whether the studied systems have tests, which RQ2 asks about, and (b) the primary oracle, the one behind the headline result. Codes that record only presence cannot support "relies on" (Section 5.3) or G5.]

5 Results

Counts below refer to the 49 papers of the main corpus; the two cross-language equivalence-detection papers (I3) are reported at the end of Section 5.4. A paper can use several oracles and several migration kinds, so columns do not sum to 49.

5.1 Corpus overview

The corpus grows quickly: 8 papers are from 2024, 14 from 2025 and 27 from 2026, or 0.68%, 0.98% and 1.45% of the papers scanned in each year (Table 2). The 2024 share may be slightly low, since 61 ICSE 2024 papers had no abstract and were matched on their titles only.

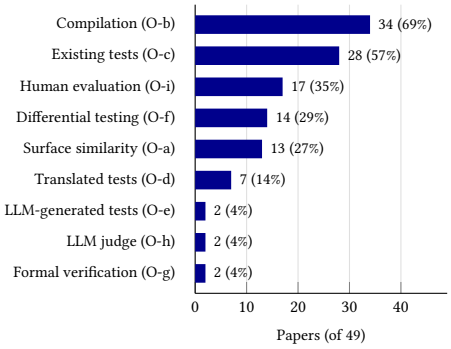


Figure 2: [revised] Oracles used by the 49 main-corpus papers. A paper can use several oracles. An oracle is counted when the paper reports its result on the final outputs.

Most papers are research-track (28), followed by industry/SEIP (9), journal-first (5), NIER (3), posters (2), and one each in demonstrations and Ideas, Visions and Reflections (IVR). We call the NIER, IVR, demonstration and poster papers short papers. Translation between languages dominates (XL, 33 papers), followed by library migration (LIB, 12), version migration (VER, 3), framework migration (FW, 3) and dialect translation (DIA, 2); 5 papers migrate legacy systems, as defined in Section 2. Rust is a target language in 13 papers. The LLM is the proposed technique in 32 papers, the evaluated subject in 9, and both in 4; in the remaining 4 it is only a baseline (3, two of which propose a smaller neural model) or part of the evaluation tooling (1).

5.2 RQ1: Which oracles are used?

Compilation and existing tests dominate; formal verification, LLM-generated tests and LLM judges are rare (Figure 2). We count an oracle when a paper reports its result on the final outputs, so the counts record which oracles papers use, not which ones their conclusions rest on.

Most papers combine oracles: 9 use one, 17 use two, 17 use three, and 6 use four or more, up to five in RustAssure [4]. Six of the 28 papers with existing tests report no compile rate, although their code must compile to run. Nine papers use no oracle that executes code (none of O-c to O-f): they judge outputs by compilation, similarity, a human or an LLM. Five rely on a single oracle that does not run the code: 2 report only BLEU or CodeBLEU [13, 29], one only an LLM judge [8], and 2 only human judgment [28, 51]. **[TODO: O-b rule still open (reply to the response, V2). If RustRepoTrans and TransGraph lose O-b, compilation becomes 32 papers (65%), 8 of the 28 papers with existing tests report no compile rate, and 11 papers use one oracle. If CAST gains O-b, compilation becomes 35 (71%) and 5 of the 28 report no compile rate. Update Figure 2, Tables 5 and 6 and this paragraph once the team decides.]**

Table 5 shows which oracles are combined. Compilation accompanies most other oracles: 22 of the 28 papers with existing tests also use compilation as an oracle, and so do 12 of the 17 with human

Table 5: [new] [revised] Oracle combinations in the main corpus. Each cell counts the papers that use both oracles; the diagonal gives the papers that use each oracle.

	O-a	O-b	O-c	O-d	O-e	O-f	O-g	O-h	O-i
O-a	13								
O-b	6	34							
O-c	7	22	28						
O-d	0	7	5	7					
O-e	1	1	1	0	2				
O-f	1	12	4	3	0	14			
O-g	0	2	0	1	0	2	2		
O-h	0	1	0	0	0	0	0	2	
O-i	4	12	6	3	0	5	1	1	17

evaluation. Differential testing seldom appears next to existing tests (4 of 14 papers), and in 9 of the 14 no test-suite oracle (O-c, O-d, O-e) is used. Both papers with formal verification also run differential tests.

5.3 RQ2: Where do oracles come from, and in which contexts?

Oracle use differs by context in this corpus (Table 6). Most groups are small, so patterns in groups of five papers or fewer are illustrative only.

- **No legacy paper uses existing tests.** Two of the 5 legacy papers run the original system as a differential oracle [17, 23], one generates tests with an LLM from an extracted specification [7], one uses an LLM judge [8], and one compares the generated Java with the expected code by hand [51]. This is the one empty cell that chance does not easily explain: with 28 of the 49 papers using existing tests, about 3 legacy papers would be expected to use them. With five papers, all in the industry/SEIP or NIER tracks, legacy cannot be separated from track, and the result depends on our definition: if the three papers that call their C or Java sources legacy were flagged [11, 49, 52], 2 of 8 legacy papers would use existing tests.
- **Library migration often includes a human check.** Nine of the 12 LIB papers include human evaluation, against 6 of 33 XL papers, but only 3 of these nine judge behavior without also running the code [1, 28, 35]. Several human checks cover only part of the outputs, such as a 20% sample of the patches that pass the tests [16] or the 24 cases that have a developer's fix [54]. No LIB paper uses LLM-generated tests or formal verification.
- **Both formal-verification papers target Rust** [4, 57]; with two papers, this is an observation rather than a pattern.
- **LLM-based final-output oracles appear only in 2026.** The 2 papers with LLM-generated tests [7, 59] and the 2 with LLM judges [8, 35] are all from 2026. Earlier papers use LLM-generated tests only inside the technique [29, 58], and with four papers the timing may be chance.
- **All 5 journal-first papers use existing tests**, and none uses human evaluation.
- **Zero cells** mark combinations that no paper uses: legacy with similarity, existing, translated or formal-verification oracles; library migration with LLM-generated tests or formal verification; dialect translation with any test-suite oracle; and short papers with translated tests, generated tests, formal verification or LLM judges. Except for legacy with existing tests, each of these cells would hold at most 1.3 papers if oracle use did not depend on the group, so an empty cell says little on its own.

Where the ground truth comes from. Table 7 breaks down the four execution-based oracles by the origin of their ground truth; we do not code the origin of the other oracles. Existing tests split almost evenly: 14 papers use tests built for a benchmark of curated problems or reference translations, 12 use the test suite that a real project's developers wrote, and 2 use both [15, 25]. Among the 14 LIB and VER papers, 7 use existing tests, and 6 of these 7 rerun the client project's own suite after the change [16, 26, 37, 40, 47, 54]. In translation between languages, benchmark tests are more common: 12 papers, against 6 with project suites and 2 with both.

Translated tests are mostly translated by the system under evaluation (5 of 7). In 4 of these papers a translation error can therefore affect the code and its tests alike; in IntentTester the migrated tests are themselves the output under evaluation, so a pass shows that a migrated test passes on the target library, not that it still checks what the original test checked [19]. In 3 papers the authors translated tests by hand, in one of them alongside tests translated by its system [4, 25, 43]. Only 2 papers use LLM-generated tests in the sense of Table 1, where the expected outputs come from an LLM or a specification: one derives its tests from the same specification it uses to generate the code [7], and one does not say where the expected outputs come from, which we code as O-e [59]. Tests that an LLM writes but whose expected outputs come from running the original are coded as differential testing (Section 2). Differential testing takes its inputs from fuzzing, property-based testing or symbolic execution in 5 papers, reuses existing tests or benchmark data in 3, uses LLM-generated inputs in 3 [3, 25, 33], compares curated samples or rendered screens in 2, and does not describe its inputs in 2; one paper has two input origins [25].

5.4 RQ3: Threats to oracle reliability

Oracle disagreement. Thirty-three papers apply two or more oracles to the same outputs and report numbers under each in the text or tables. A few more apply several oracles but report them only as a combined rate [47], a pass/fail summary [7, 31], a chart [34], or not separately [40]. The largest and most consistent gap is between compiling and behaving correctly: 18 papers report both a compile rate (or the rate of another well-formedness check) and a behavioral pass rate for the same outputs, and in every configuration plotted in Figure 3 behavior trails compilation, by 3 to 98 percentage points.

Other disagreements involve surface similarity and test strength (Table 8). Besides the 18 papers in Figure 3 and 6 more in Table 8, the other 9 papers report numbers under several oracles in other forms: counts of compile errors next to test pass rates [49], breakdowns of failures by stage [42, 43], a compile rate per function next to equivalence per return value [4], bugs found by each checking stage [3], build errors next to diff similarity [5], rendered-screen similarity next to structural similarity [18], a manual review of outputs that failed an AST match [1], and a syntax-check rate next to a correctness rate from matching and manual review [26].

Table 6: [revised] Oracle use by context. Each cell counts papers in the group that use the oracle; *n* is the group size. IVR: Ideas, Visions and Reflections.

Group	*n*	O-a	O-b	O-c	O-d	O-e	O-f	O-g	O-h	O-i
Legacy	5	0	1	0	0	1	2	0	1	1
Not legacy	44	13	33	28	7	1	12	2	1	16
XL	33	7	22	20	6	2	11	2	1	6
LIB	12	4	9	6	2	0	1	0	1	9
VER	3	1	2	1	0	0	1	0	0	1
FW	3	1	3	1	0	1	1	0	0	1
DIA	2	1	1	0	0	0	1	0	0	2
Research	28	8	19	15	6	1	8	2	1	13
Industry/SEIP	9	2	7	5	1	1	3	0	1	2
NIER, IVR, demo, poster	7	2	4	3	0	0	2	0	0	2
Journal-first	5	1	4	5	0	0	1	0	0	0
2024	8	3	5	5	1	0	2	0	0	1
2025	14	3	10	8	3	0	4	2	0	6
2026	27	7	19	15	3	2	8	0	2	10

Table 7: [new] [revised] Where the ground truth of execution-based oracles comes from (main corpus). Per-paper codes are in Appendix A. *One paper uses both translated-test origins, so the O-d rows sum to 8. One differential-testing paper has two input origins, so the O-f rows sum to 15. Origin is coded only for these four oracles.

Oracle and origin	Papers
O-c Existing tests (28)	
Built for a benchmark (curated problems, reference translations)	14
A real project's own test suite	12
Both	2
O-d Translated tests (7)	
Translated by the system under evaluation	5
Translated by hand by the authors*	3
O-e LLM-generated tests (2)	
Derived from the specification that also generates the code	1
Source of expected outputs not stated	1
O-f Differential testing (14)	
Inputs from fuzzing, property-based testing or symbolic execution	5
Inputs from existing tests or benchmark data	3
Inputs generated by an LLM	3
Curated samples or rendered screens	2
Inputs not described	2

Three patterns recur: compiling says little about equivalence; surface similarity can rank systems in the opposite order to execution; and tests and fuzzing checks miss faults that mutation analysis, verification or human review find.

Limitations that studies report. We grouped the oracle limitations that papers state about their own evaluation (Table 9).

Test weakness is the most common main limitation (15 papers), even though existing tests remain the second most used oracle (28 papers). Nine papers state no limitation of their own oracle, including both BLEU-only papers and the paper that relies only on an LLM judge. Oracle gaming is a new theme: a model rewarded for compiling emits trivial code such as `panic!` [15], an LLM rewrites code until tests pass [49], and benchmark agents game the tests, which FreshBrew detects with a coverage threshold [37].

Oracles reused inside the technique. Although we code only the oracle that judges the final output (Section 4.5), in some papers the same oracle also selects or repairs the outputs it later judges. InterTrans stops its search as soon as a candidate passes the benchmark tests and reports accuracy on those same tests [36]; SmartC2Rust runs the same tests in its repair loop and as its final measure [49]; and RISE uses its differential check both as a validity step inside the technique and as the evaluation [55]. In these 3 papers the reported pass rate shows that the output satisfies the oracle it was selected against, not that it is equivalent; a held-out oracle would separate the two. Two further papers train their models against the same kind of oracle they report, though not necessarily on the evaluated problems: TRAVEL rewards its model with LLM-generated test components of the kind it also evaluates with [33], and DGIR, in an ablation that rewards compilation only, shows where this leads (Figure 3) [15]. We flagged these cases during extraction rather than coding them systematically, so there may be more.

Cross-language equivalence detection (I3). Two papers form the separate tier. One measures how well LLMs judge whether code in two languages is equivalent: GPT-3.5-Turbo reaches F1 0.99 on XLCoST but 0.82 on CodeNet [39], which is direct evidence on LLMs as oracles (O-h). The other checks equivalence formally between C++ or PyTorch specifications and Chisel or Verilog implementations through MLIR, without an LLM [61]: a kind of formal cross-language oracle that the software translation papers in our corpus barely use.

Table 8: Other oracle disagreements reported on the same outputs. TerraMod and VERT also appear in Figure 3. CA: computational accuracy, the share of outputs that pass the tests.

Paper	Oracles compared	Result
UniTrans [58]	Tests (CA) vs exact match	87.92% vs 18.04% (GPT-3.5)
CAST [14]	Tests (CA) vs exact match	69.72% vs 16.23% (GPT-4o)
TransAgent [59]	Tests (CA) vs CodeBLEU	TransCoder Python to Java: CA 2.4%, CodeBLEU 40.1
TerraMod [22]	Execution (Pass@1) vs CrystalBLEU	LLaMA-4-Maverick: baseline 44% / 0.78 vs TerraMod 76% / 0.66; the two metrics rank the systems in opposite order (one of four models)
VERT [57]	Compile, property-based tests, bounded and full verification	C++, 569 programs: 539, 295, 233 and 19 pass
MBTA [21]	Tests (CA) vs mutation-based score	168 programs reach perfect CA, yet TransCoder mistranslates at least one of their mutants
BDUpdater [54]	Existing tests vs developer's fix	1 of 24 adaptations passes the tests but is not equivalent
Code hints [11]	Fuzzing check vs manual review	Symbolic engine without hints: 23 candidates pass the check, 7 are correct

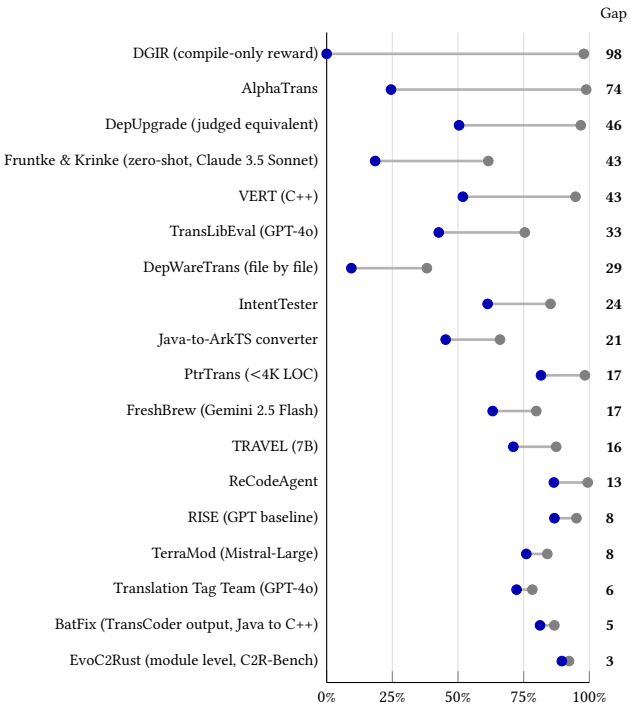


Figure 3: Share of outputs passing compilation or another well-formedness check (gray) versus behavioral pass rate (blue) in the 18 papers that report both for the same outputs, with the gap in percentage points. Numbers are as reported in each paper, for the configuration named in parentheses and otherwise for the paper's own system; the DGIR row is its compile-only-reward ablation, the RISE row its GPT-based LLM baseline, the DepWareTrans row a file-by-file baseline, and the BatFix row the translations before repair. The behavioral oracle differs by paper.

Table 9: Oracle limitations that papers state about their own evaluation. Each paper is counted once, under its main theme.

Theme	Papers
Weak or few tests, so passing tests does not imply equivalence	15
Incomplete differential, fuzzing, symbolic or bounded checks	6
Subjective human evaluation, or no agreement measured	4
The LLM games the oracle: optimizes for compiling or passing tests	3
Oracle derived from the same source as the output (tests and code generated from one specification)	3
Misleading surface similarity	3
Compiling does not imply correct behavior	2
No executable oracle available	2
A limitation of the oracle unrelated to equivalence (e.g., readability)	2
No limitation of the paper's own oracle stated	9

6 Discussion

6.1 RQ4: Gaps between evaluation practice and real migration

Real migration projects, legacy ones above all, often have no usable tests, cannot re-check every output by hand, and need evidence that holds beyond a benchmark. Measured against these needs, the map shows six gaps.

G1. Legacy settings are rare and appear only in the industry/SEIP and NIER tracks. Only 5 of the 49 papers migrate a legacy system, all in these tracks, and none of them uses existing tests. Two run the original system as a differential oracle; the other 3 rely on generated tests, an LLM judge or a person (Section 5.3). **[Hà Cao:** Depends on the legacy definition (see Section 2): with EvoC2Rust, SmartC2Rust and Code hints flagged legacy, 2 of 8 legacy papers use existing tests.**]**

G2. Test evidence rests largely on benchmarks. Of the 28 papers that use existing tests, 14 use only tests built for benchmarks of curated problems and 2 use both kinds; in translation

between languages benchmark tests outnumber project test suites 12 to 6. A pass rate on such tests says little about a repository whose own tests are missing or weak.

G3. Derived oracles are seldom independent of the output. In 5 of the 7 papers with translated tests, the evaluated system translated them, so in 4 of them a translation error can affect the code and its tests alike; one paper generates tests and code from the same specification; and in at least 3 papers the reported oracle also selects or repairs the outputs it judges (Section 5.4).

G4. Judgment-based oracles go unvalidated. Human evaluation appears in 18 papers, but only 4 report agreement between raters [16, 25, 26, 56]. LLM judges appear in 3 papers, and one industrial paper relies on an LLM judge alone [8]. **[Hà Cao:** 2 if ReCodeAgent's O-h is recoded (its LLM judges the translated tests, not the translation).**]**

G5. Compilation is reported as if it were evidence. Thirty-three papers use compilation as an oracle; in the 18 that report a compile rate and a behavioral pass rate for the same outputs, behavior trails compilation by 3 to 98 percentage points in the configurations shown in Figure 3. **[Hà Cao:** From the RQ1 data: 32 of the 33 papers with compilation also use a behavioral, human or LLM oracle; only CodePlan reports compilation without one. "Reported as if it were evidence" needs either a primary-oracle code (papers whose headline metric is compilation, e.g., DepUpgrade) or rewording.**]**

G6. Oracle limitations go unstated. Nine papers state no limitation of their own oracle, including all three that rely on a single weak oracle. **[Hà Cao:** Depends on how "weak" is defined (Section 5.2): counting the human-only papers (Pig, PL/I), it is 4 of 5.**]**

What to report and what to study next. The checklist in Section 6.3 addresses what papers should report. Four research directions follow from the gaps: benchmarks with a setting that withholds tests, in which equivalence must be shown against a runnable original, as legacy projects require (G1, G2); ways to validate translated and generated tests before trusting them, for example by mutation analysis [21] or by checking them against the original (G3); LLM judges calibrated against execution, with agreement reported by task difficulty, given that the F1 score of LLMs judging equivalence already drops between benchmarks [39] (G4); and evaluation protocols that keep one oracle hidden from the technique, so that optimizing against the oracle cannot inflate the result (G3, G5).

6.2 Implications

For researchers. A compile rate is not evidence of equivalence (G5). Results should lead with a behavioral oracle and report compilation only as a filter. Where tests are the oracle, their strength matters as much as the pass rate: how many tests per sample, what they cover, and whether they were translated, generated or pre-existing. Test weakness is already the most common main limitation (15 papers, Table 9); reporting test strength would let readers judge it.

For benchmark builders. Benchmarks shape the oracle a field uses. Surface similarity (O-a) still appears in 14 papers and ranks systems in the opposite order to execution in at least one [22]. Benchmarks should ship executable oracles with known coverage, and guard against gaming as FreshBrew does with a coverage gate [37]. The zero cells of Table 6 point to benchmarks that do not yet exist: legacy migrations with existing or translated test suites, dialect pairs with test suites, and library migrations evaluated with generated tests or formal verification. **[Hà Cao:** Most zero cells are expected by chance given the group sizes (Section 5.3); only legacy × existing tests is informative. Soften to "may point to".**]** Even where library migrations have tests, they rarely cover the change: in one study only 27 of 314 migrations had tests covering the changed code [26].

For practitioners migrating real systems. No legacy paper in our corpus uses existing tests, and the corpus shows what replaces them: running the original system as a differential oracle, tests generated from an extracted specification, an LLM judge or a human. These differ in trust. A differential oracle uses the original program as ground truth. Tests and code generated from the same specification can only show consistency with that specification, as one industrial paper states itself [7]. LLM judges are rarely validated: the one paper that relies only on an LLM judge mentions a human check but reports no agreement figure [8], and Moumoula et al. show the F1 score of an LLM judging cross-language equivalence dropping from 0.99 to 0.82 between benchmarks [39]. Out-of-scope work points the same way: in function-level migration, passing parity tests did not guarantee preserved behavior [30].

6.3 A checklist for oracle provenance

We propose that papers on LLM-based translation and migration report, for each oracle:

(1) What it judges (well-formedness, behavior on given inputs, equivalence for all inputs, similarity, a person's or model's verdict).
(2) Where it comes from: existing, translated with the code, generated (by whom), derived from a specification, or obtained by running the original.
(3) How strong it is: number of tests or inputs per sample, coverage, input-generation budget, verification bounds.
(4) Whether it is independent of the output: produced by the same model, prompt or specification, or used inside the technique (repair loop, reward, filter).
(5) Its numbers alongside every other oracle applied to the same outputs.
(6) For human or LLM judges: who or which model, the protocol, and agreement with a ground truth or between raters.
(7) Its known limitations in this setting.

7 Threats to Validity

Construct validity. Classifying oracles and contexts requires judgment, for instance when a compiler serves both as a filter and as a reported metric. We mitigate this with a written codebook, by coding only oracles that judge the final output, and by discussing hard cases until agreement. Some codes record what papers write rather than what they do. A paper that states no oracle limitation may still be aware of one, so the 9 papers in that row of Table 9 measure reporting, not awareness. The origin codes in Table 7 depend on

how fully papers describe their tests and inputs, which is why two differential-testing papers fall under "not described". We take every reported number as published, without re-running any experiment, so the gaps in Figure 3 compare reported configurations. The configuration we plot for each paper (named in the figure) affects the size of its gap. Some behavioral rates also count outputs that no test covers as failures [24, 60], and ReCodeAgent's is a rate over tests rather than over outputs [25], so the gaps are indicative rather than exact. Journal-first papers are counted under the conference and year that presented them. **[TODO: report the calibration on five shared papers.]** **[Hà Cao:** Also name the coding choices that drive RQ1/RQ2: the O-e/O-f boundary, the narrow legacy definition, and that oracle codes record presence, not reliance.**]**

Internal validity. A keyword search can miss papers. Against papers we knew in advance, the string retrieved 39 of 40. For 80 papers without an abstract, the keyword string could be matched against the title only; the poster tracks of ICSE 2025, FSE 2026 and ASE 2026 had no paper list at the time of the scan; and 3 papers available only behind a paywall were excluded under E6. Exclusions could be decided on title and abstract, while every inclusion required the full text (Figure 1); an early exclusion can therefore miss a relevant paper, and the recall check bounds this risk only for the papers we knew in advance. Extraction was drafted from full text and the reported numbers were re-checked against the papers, but some versions read were preprints rather than the published version; we mark these in the data.

External validity. The map reflects the main proceedings of ICSE, FSE and ASE from 2024 to 2026, without co-located workshops, where much legacy and migration work appears; this area also moves fast on arXiv, so the newest work may be missing. Our conclusions are limited to practice at these venues. The corpus also mixes full papers with seven short ones (NIER, posters, a demonstration and an Ideas, Visions and Reflections paper), which describe their evaluation in less detail and may therefore understate the oracles they use.

Reliability. With 49 papers, we report counts and do not test differences between groups; the trend over years rests on three years, the last of which (2026, 27 papers) is larger than the first two together. **[Hà Cao:** Results still phrase several small-cell patterns as rules (Section 5.3); hedge them there, or note here that patterns in groups of n≤5 are illustrative only.**]** Every count can be recomputed from the replication package (Section 8). The data are a snapshot as of October 23, 2026.

8 Conclusion and Data Availability

We mapped how 49 papers at ICSE, FSE and ASE from 2024 to 2026 decide whether LLM-translated or migrated code behaves like the original. Compilation and existing tests dominate, and legacy settings fall back on differential runs, generated tests, LLM judges or people. Of the 33 papers that report several oracles on the same outputs, 18 report both a compile and a behavioral rate, and there behavior trails compilation by up to 98 percentage points. The gaps to real migration are clear: legacy settings are rare, test evidence rests largely on benchmarks, derived oracles are seldom independent of the output, judgments by people or LLMs are rarely validated, compilation is reported as if it were evidence, and 9 papers state no limitation of their own oracle. Before we trust LLMs to translate the software that runs the world, we must agree on how to check their work; the provenance checklist in Section 6.3 is a first step.

Data availability. The replication package contains the search log with dates and counts for every proceedings volume, the codebook, every screening decision with its reason and evidence basis, the extraction form for all 51 papers, the coded data behind every table and figure, and a script that recomputes every count: **[TODO: GitHub link, or switch to the non-public version below]**.

References

[1] Axel Allain, Aymeric Blot, Djamel Eddine Khelladi, and Mathieu Acher. 2026. Code Transformation Rule Synthesis using LLMs: Potential and Limits. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2639–2651. doi:10.1145/3832783.3837504

[2] Aylton Almeida, Marco Tulio Valente, and Laerte Xavier. 2026. Using Copilot Agent Mode to Automate Library Migration: A Quantitative Assessment. In *Proceedings of the 2026 International Workshop on Agentic Engineering*. ACM, 135–139. doi:10.1145/3786167.3788411

[3] Yeaseen Arafat and Stefan Nagy. 2026. PROGnosticator: Testing Source-to-Source Code Translators via Construct-Oriented Fuzzing. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 876–898. doi:10.1145/3797135

[4] Yubo Bai and Tapti Palit. 2025. RustAssure: Differential Symbolic Testing for LLM-Transpiled C-to-Rust Code. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 534–546. doi:10.1109/ase63991.2025.00051

[5] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C., Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. 2024. CodePlan: Repository-Level Coding using LLMs and Planning. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 675–698. doi:10.1145/3643757

[6] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 5 (2015), 507–525. doi:10.1109/tse.2014.2372785

[7] Andreas Biesdorf, Eckhard Seibert, Nicole Wengatz, Bart Carpels, and Tom Schoonbaert. 2026. IP-Preserving Enterprise System Modernization via LLM-Driven Specification Extraction and Multi-target Code Generation. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 4463–4468. doi:10.1145/3832783.3834533

[8] Venkatesan T. Chakaravarthy, Anamitra Roy Choudhury, Dinesh Garg, Vini Kanvar, Shivmaran S. Pandian, Aditya Raghuvanshi, Yogish Sabharwal, and Amith Singhee. 2026. Enterprise-Scale COBOL-to-Java Translation: LLMs Augmented with Program Analysis. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*. ACM, 775–785. doi:10.1145/3786583.3786915

[9] Sivajeet Chand, Alexander Pretschner, Steve Haupt, Derui Zhu, and Sushant Kumar Pandey. 2026. DepWareTrans: Dependency-Aware Incremental Repository Migration across Co-executable Languages. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 4164–4174. doi:10.1145/3832783.3834505

[10] Xiang Chen, Jiacheng Xue, Xiaofei Xie, Caokai Liang, and Xiaolin Ju. 2025. A Systematic Literature Review on Neural Code Translation. arXiv preprint arXiv:2505.07425. https://arxiv.org/abs/2505.07425

[11] Cristina David, Pascal Kesseli, Daniel Kroening, and Hanliang Zhang. 2025. Quantifying the Benefits of Code Hints for Refactoring Deprecated Java APIs. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. ACM, 444–455. doi:10.1145/3696630.3728567

[12] Simantika Bhattacharjee Dristi and Matthew B. Dwyer. 2026. A Differential Fuzzing-Based Evaluation of Functional Equivalence in LLM-Generated Code Refactorings. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*. ACM, 1708–1712. doi:10.1145/3803437.3806717

[13] Yali Du, Hui Sun, and Ming Li. 2024. A Joint Learning Model with Variational Interaction for Multilingual Program Translation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1907–1918. doi:10.1145/3691620.3695553

[14] Yali Du, Hui Sun, and Ming Li. 2025. Post-Incorporating Code Structural Knowledge Into Pretrained Models via ICL for Code Translation. *IEEE Transactions on Software Engineering* 51, 11 (2025), 3038–3055. doi:10.1109/tse.2025.3605768

[15] Jia Feng, Wenjie Gan, Cuiyun Gao, Chaozheng Wang, Feng Luo, Xin Xia, Ge Li, and Kui Liu. 2026. Dependency-Guided Repository-Level C-to-Rust Translation

with Reinforcement Alignment. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*. ACM, 954–965. doi:10.1145/3803437.3805266

[16] Lukas Fruntke and Jens Krinke. 2025. Automatically Fixing Dependency Breaking Changes. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2146–2168. doi:10.1145/3729366

[17] Kavyasri Gajula, Phanindra Kodela, and Sridhar Chimalakonda. 2026. K-COBAgent: A Knowledge-Driven Multi-agent Framework for COBOL Modernization. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 4631–4635. doi:10.1145/3832783.3834560

[18] Yi Gao, Xing Hu, Xiaohu Yang, and Xin Xia. 2026. GUIMigrator: Semantics-Preserving Transpilation from Android XML to Compose and SwiftUI. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 4737–4759. doi:10.1145/3808215

[19] Yi Gao, Ziyuan Zhang, Xing Hu, Xiaohu Yang, and Xin Xia. 2026. IntentTester: Intent-Driven Multi-agent Framework for Cross-Library Test Migration. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 1515–1535. doi:10.1145/3797096

[20] Lars Gröninger, Beatriz Souza, and Michael Pradel. 2025. ChangeGuard: Validating Code Changes via Pairwise Learning-Guided Execution. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 936–956. doi:10.1145/3715760

[21] Giovani Guizzo, Jie M. Zhang, Federica Sarro, Christoph Treude, and Mark Harman. 2023. Mutation Analysis for Evaluating Code Translation. *Empirical Software Engineering* 29, 1 (2023). doi:10.1007/s10664-023-10385-w

[22] Pranjal Gupta, Pooja Aggarwal, Brent Paulovicks, Prateeti Mohapatra, Rong Lee, and Vadim Sheinin. 2026. Automating Terraform Code Migration through Provider Evolution Knowledge. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*. ACM, 211–215. doi:10.1145/3786582.3786841

[23] Sandeep Hans, Atul Kumar, Toshiaki Yasue, Kouichi Ono, Saravanan Krishnan, Devika Sondhi, Fumiko Satoh, Gerald Mitchell, Sachin Kumar, and Diptikalyan Saha. 2025. Automated Testing of COBOL to Java Transformation. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. ACM, 227–237. doi:10.1145/3696630.3728548

[24] Ali Reza Ibrahimzada, Kaiyao Ke, Mrigank Pawagi, Muhammad Salman Abid, Rangeet Pan, Saurabh Sinha, and Reyhaneh Jabbarvand. 2025. AlphaTrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2454–2476. doi:10.1145/3729379

[25] Ali Reza Ibrahimzada, Brandon Paulsen, Daniel Kroening, and Reyhaneh Jabbarvand. 2026. ReCodeAgent: A Multi-agent Workflow for Language-Agnostic Translation and Validation of Large-Scale Repositories. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2409–2421. doi:10.1145/3832783.3837486

[26] Mohayeminul Islam, Ajay Kumar Jha, May Mahmoud, Ildar Akhmetov, and Sarah Nadi. 2025. An Empirical Study of Python Library Migration Using Large Language Models. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 867–879. doi:10.1109/ase63991.2025.00077

[27] Mingsheng Jiao, Tingrui Yu, Xuan Li, Guanjie Qiu, Xiaodong Gu, and Beijun Shen. 2023. On the Evaluation of Neural Code Translation: Taxonomy and Benchmark. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1529–1541. doi:10.1109/ase56229.2023.00114

[28] Miryeong Kang, Wonseok Oh, Gabin An, and Hakjoo Oh. 2026. Pig: Leveraging Large Language Models for Python Library Migrations. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 967–988. doi:10.1145/3797072

[29] Rabimba Karanjai, Sam Blackshear, Lei Xu, and Weidong Shi. 2025. Collaboration Is All You Need: LLM Assisted Safe Code Translation. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. ACM, 671–675. doi:10.1145/3696630.3728521

[30] Simarjot Khanna. 2026. Passing Parity Is Not Preserving Behaviour: Oracle Escape in Function-Level LLM Code Migration. In *Proceedings of the 2nd International Workshop on AI for Software Modernization*. ACM, 61–68. doi:10.1145/3843775.3844550

[31] Xingpei Li, Zhijie Jiang, Zhouyang Jia, Yuanliang Zhang, Zheng Si, Haifang Zhou, and Shanshan Li. 2025. Enhancing Deep Learning Transpilation with LLMs: Automation and Semantic Consistency. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. ACM, 1238–1239. doi:10.1145/3696630.3731431

[32] Runlin Liu, Yuhang Lin, Yunge Hu, Zhe Zhang, and Xiang Gao. 2024. LLM-Based Java Concurrent Program to ArkTS Converter. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2403–2406. doi:10.1145/3691620.3695362

[33] Feng Luo, Jiachen Liu, Cuiyun Gao, Jia Feng, and Kui Liu. 2026. Towards Reliable C-to-Rust Translation with Rule-Guided Reasoning and Reinforcement Learning. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 4469–4480. doi:10.1145/3832783.3834534

[34] Yang Luo, Richard Yu, Fajun Zhang, Ling Liang, and Yongqiang Xiong. 2024. Bridging Gaps in LLM Code Translation: Reducing Errors with Call Graphs and Bridged Debuggers. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2448–2449. doi:10.1145/3691620.3695322

[35] Xiangxi Ma, Yifan An, Wentong Tian, Xuanqi Wang, Qingao Dong, Xiang Gao, and Hailong Sun. 2026. DepUpgrade: Automating Dependency Upgrade through State-Path Exploration. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1844–1856. doi:10.1145/3832783.3837442

[36] Marcos Macedo, Yuan Tian, Pengyu Nie, Filipe R. Cogo, and Bram Adams. 2025. INTERTRANS: Leveraging Transitive Intermediate Translations to Enhance LLM-Based Code Translation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 1153–1164. doi:10.1109/icse55347.2025.00236

[37] Victor May, Diganta Misra, Yanqi Luo, Anjali Sridhar, Justine Gehring, and Silvio Soares Ribeiro. 2026. FreshBrew: A Benchmark for Evaluating AI Agents on Java Code Migration. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*. ACM, 944–955. doi:10.1145/3744916.3773214

[38] Nausheen Mohammed, Akash Lal, Aseem Rastogi, Rahul Sharma, and Subhajit Roy. 2025. LLM Assistance for Memory Safety. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 1717–1728. doi:10.1109/icse55347.2025.00023

[39] Micheline Bénédicte Moumoula, Abdoul Kader Kaboré, Jacques Klein, and Tegawendé F. Bissyandé. 2025. The Struggles of LLMs in Cross-Lingual Code Clone Detection. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 1023–1045. doi:10.1145/3715764

[40] Stoyan Nikolov, Daniele Codecasa, Anna Sjövall, Maxim Tabachnyk, Satish Chandra, Siddharth Taneja, and Celal Ziftci. 2025. How is Google Using AI for Internal Code Migrations?. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 481–492. doi:10.1109/icse-seip66354.2025.00048

[41] Vikram Nitin, Rahul Krishna, Luiz Lemos do Valle, and Baishakhi Ray. 2026. C2SaferRust: Transforming C Projects Into Safer Rust With NeuroSymbolic Techniques. *IEEE Transactions on Software Engineering* 52, 2 (2026), 618–630. doi:10.1109/tse.2025.3641486

[42] Guangsheng Ou, Mingwei Liu, Yuxuan Chen, Yanlin Wang, Xin Peng, and Zibin Zheng. 2025. RustRepoTrans: Repository-level Context Code Translation Benchmark Targeting Rust. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 610–622. doi:10.1109/ase63991.2025.00057

[43] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ACM, 1–13. doi:10.1145/3597503.3639226

[44] Brent Pappas, Joseph Zalusky, Zachary Burkett, and Paul Gazzillo. 2026. Translation Tag Team: Formal Rules and LLMs Translate More Macros Together Than Apart. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1457–1469. doi:10.1145/3832783.3837412

[45] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. 2015. Guidelines for Conducting Systematic Mapping Studies in Software Engineering: An Update. *Information and Software Technology* 64 (2015), 1–18. doi:10.1016/j.infsof.2015.03.007

[46] Daniel Ramos, Inês Lynce, Vasco Manquinho, Ruben Martins, and Claire Le Goues. 2024. BatFix: Repairing Language Model-Based Transpilation. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 1–29. doi:10.1145/3658668

[47] Frank Reyes, May Mahmoud, Federico Bono, Sarah Nadi, Benoit Baudry, and Martin Monperrus. 2026. Byam: Fixing Breaking Dependency Updates with Large Language Models. *Empirical Software Engineering* 31, 4 (2026). doi:10.1007/s10664-026-10835-1

[48] Aymane Rgaguena, Imane Chlioui, and Maryam Radgui. 2025. Generative AI for Code Translation: A Systematic Mapping Study. *Engineering Proceedings* 112, 1 (2025), 33. doi:10.3390/engproc2025112033 Presented at ICATH 2025.

[49] Momoko Shiraishi, Yinzhi Cao, and Takahiro Shinagawa. 2026. SmartC2Rust: Iterative, Feedback-Driven C-to-Rust Translation via Large Language Models for Safety and Equivalence. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*. ACM, 980–991. doi:10.1145/3744916.3773259

[50] Hui Sun, Anderson Uchôa, Rohit Gheyi, and Wesley K. G. Assunção. 2026. Evaluating Language Models on Cross-Language Code Functional Equivalence. arXiv preprint arXiv:2608.23961. https://arxiv.org/abs/2608.23961 ESEM 2026.

[51] Takaaki Tateishi and Yasuharu Katsuno. 2026. Translating PL/I Macro Procedures into Java Using Automatic Templatization and Large Language Models. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*. ACM, 46–50. doi:10.1145/3786582.3786808

[52] Chaofan Wang, Tingrui Yu, Beijun Shen, Jie Wang, Dong Chen, Wenrui Zhang, Yuling Shi, Chen Xie, and Xiaodong Gu. 2026. EvoC2Rust: A Skeleton-guided Framework for Project-Level C-to-Rust Translation. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*. ACM, 94–105. doi:10.1145/3786583.3786856

[53] Qilong Wu, Taoran Li, Tianyang Zhou, and Varun Chandrasekaran. 2025. SoK: Understanding (New) Security Issues Across AI4Code Use Cases. arXiv preprint arXiv:2512.18456. https://arxiv.org/abs/2512.18456

[54] Yifan Xia, Chengwei Liu, Zifan Xie, Lyuye Zhang, Peiyu Liu, Kangjie Lu, Yang Liu, Wenhai Wang, and Shouling Ji. 2026. Break to Adapt: Knowledge-Based Updates of Breaking Dependencies in JavaScript. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 2442–2464. doi:10.1145/3808116

[55] Xudong Xie, Yuwei Zhang, Wensheng Dou, Yu Gao, Ziyu Cui, Jiansen Song, Rui Yang, and Jun Wei. 2026. RISE: Rule-Driven SQL Dialect Translation via Query Reduction. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*. ACM, 968–979. doi:10.1145/3744916.3773257

[56] Pengyu Xue, Kunwu Zheng, Zhen Yang, Yifei Pei, Linhao Wu, Jiahui Dong, Xiapu Luo, Yan Xiao, Fei Liu, Yuxuan Zhang, Xiran Lyu, Xianhang Li, Xuanyu Zhu, and Chengyi Wang. 2026. TransLibEval: Demystify Large Language Models' Capability in Third-Party Library-Targeted Code Translation. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 920–942. doi:10.1145/3797137

[57] Aidan Z.H. Yang, Yoshiki Takashima, Brandon Paulsen, Josiah Dodds, and Daniel Kroening. 2025. VERT: Polyglot Verified Equivalent Rust Transpilation with Large Language Models. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1453–1463. doi:10.1109/ase63991.2025.00123

[58] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. 2024. Exploring and Unleashing the Power of Large Language Models in Automated Code Translation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1585–1608. doi:10.1145/3660778

[59] Zhiqiang Yuan, Weitong Chen, Hanlin Wang, Xin Peng, Zhenpeng Chen, and Yiling Lou. 2026. TransAgent: Enhancing LLM-Based Code Translation via Fine-Grained Execution Alignment. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 1582–1604. doi:10.1145/3797099

[60] Zhiqiang Yuan, Wenjun Mao, Zhuo Chen, Xiyue Shang, Chong Wang, Yiling Lou, and Xin Peng. 2026. Project-Level C-to-Rust Translation via Pointer Knowledge Graphs. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 3675–3698. doi:10.1145/3808169

[61] Jiaying Zhu, Baoqi Zhang, Mengxia Tao, Kezhi Li, Hao Yan, Qiang Xu, and Min Li. 2026. EquivFusion: Unifying Hardware Equivalence Checking from Algorithms to Netlists via MLIR. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*. ACM, 102–106. doi:10.1145/3803437.3806400

[62] Ming Zhu, Mohimenul Karim, Ismini Lourentzou, and Daphne Yao. 2024. Semi-Supervised Code Translation Overcoming the Scarcity of Parallel Code Data. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1545–1556. doi:10.1145/3691620.3695524

A Primary Studies

Table 10 lists the 51 included papers, 49 in the main corpus and 2 in the I3 tier, with the coded fields behind the frequency tables and figures of Section 5.

Table 10: Included papers and their coding. Venue: conference, year and track (Res. research, Ind. industry/SEIP, JF journal-first, IVR Ideas, Visions and Reflections). Kind: migration kind; A/B means both kinds, and +L marks a legacy source. LLM: role of the LLM (P proposed technique, E evaluated subject, P, E: both; B baseline only, T evaluation tooling, N no LLM (a smaller neural model or no model)). Oracles: letters of Table 1. Origin of execution-based oracles (Table 7): c:B benchmark tests, c:P project test suite, c:BP both; d:S translated by the system, d:H translated by hand; e:R expected outputs from running the original, e:S derived from the specification, e:U not stated; f:F fuzzing, property-based or symbolic inputs, f:T existing tests or data, f:L LLM-generated inputs, f:C curated samples or screens, f:U not described. Stated limitation: theme of Table 9.

Paper	Venue	Kind	LLM	Oracles (O-)	Origin	Stated limitation
AlphaTrans [24]	FSE 2025 Res.	XL	P	b, c, d	c:P, d:S	same source as output
BatFix [46]	ICSE 2025 JF	XL	E	b, c	c:B	weak tests
BDUpdater [54]	FSE 2026 Res.	LIB	P	c, i	c:P	weak tests
Byam [47]	ASE 2026 JF	LIB	P	b, c	c:P	other, not equivalence
C2SaferRust [41]	FSE 2026 JF	XL	P	b, c	c:P	weak tests
CAST [14]	FSE 2026 JF	XL	P, E	a, c	c:B	weak tests
COBOL-to-Java testing [23]	FSE 2025 Ind.	XL+L	E	f	f:F	incomplete checks
Code hints [11]	FSE 2025 Ind.	LIB	P	b, f, i	f:F	incomplete checks
CodePlan [5]	FSE 2024 Res.	LIB	P	a, b	–	compiling is not behaving
Collaboration [29]	FSE 2025 IVR	XL	P	a	–	none stated
DeepModelX [31]	FSE 2025 Poster	FW	P	b, c	c:B	none stated
DepUpgrade [35]	ASE 2026 Res.	LIB	P	b, h, i	–	compiling is not behaving
DepWareTrans [9]	ASE 2026 Ind.	XL	P	b, c	c:P	weak tests
DGIR [15]	FSE 2026 Ind.	XL	P	a, b, c	c:BP	oracle gaming
EvoC2Rust [52]	ICSE 2026 Ind.	XL	P	a, b, c	c:B	weak tests
FreshBrew [37]	ICSE 2026 Res.	VER	E	b, c	c:P	oracle gaming
Fruntke & Krinke [16]	FSE 2025 Res.	LIB	P	b, c, i	c:P	weak tests
Google migrations [40]	ICSE 2025 Ind.	LIB	P	b, c, d, i	c:P, d:S	subjective judgment
GUIMigrator [18]	FSE 2026 Res.	FW	B	a, b, f, i	f:C	misleading similarity
IntentTester [19]	FSE 2026 Res.	LIB/XL	P	b, d, i	d:S	subjective judgment
InterTrans [36]	ICSE 2025 Res.	XL	P	c	c:B	weak tests
IP-preserving modernization [7]	ASE 2026 Ind.	FW/XL+L	P	b, e	e:S	same source as output
Java-to-ArkTS converter [32]	ASE 2024 Demo	XL	P	b, f, i	f:C	none stated
K-COBAgent [17]	ASE 2026 NIER	XL+L	P	f	f:U	same source as output
Lost in Translation [43]	ICSE 2024 Res.	XL	E	b, c, d	c:B, d:H	weak tests
MBTA [21]	ICSE 2024 JF	XL	E	c, f	c:B, f:T	weak tests
MSA (Checked C) [38]	ICSE 2025 Res.	DIA	P	a, i	–	subjective judgment
Pig [28]	FSE 2026 Res.	LIB	P	i	–	no executable oracle
PL/I templatization [51]	ICSE 2026 NIER	XL+L	P	i	–	none stated
PROGnosticator [3]	FSE 2026 Res.	VER/XL	T	b, f	f:L	incomplete checks
PtrTrans [60]	FSE 2026 Res.	XL	P	b, e, f	e:R, f:F	weak tests
Python library migration [26]	ASE 2025 Res.	LIB	E	a, b, c, i	c:P	weak tests
ReCodeAgent [25]	ASE 2026 Res.	XL	P	a, b, c, d, e, f, h, i	c:BP, d:S, d:H, e:R, f:U	weak tests
RISE [55]	ICSE 2026 Res.	DIA	P	b, f, i	f:T	incomplete checks
Rule synthesis [1]	ASE 2026 Res.	LIB/VER	E	a, i	–	no executable oracle
RustAssure [4]	ASE 2025 Res.	XL	P, E	b, d, f, g, i	d:H, f:F	incomplete checks
RustRepoTrans [42]	ASE 2025 Res.	XL	E	b, c	c:P	none stated
Semi-supervised translation [62]	ASE 2024 Res.	XL	N	c	c:B	other, not equivalence
SmartC2Rust [49]	ICSE 2026 Res.	XL	P	b, c, d, f	c:P, d:S, f:T	oracle gaming
TerraMod [22]	ICSE 2026 NIER	LIB	P	a, b, c	c:B	misleading similarity
TransAgent [59]	FSE 2026 Res.	XL	P	a, c, e	c:B, e:U	weak tests
TransGraph [34]	ASE 2024 Poster	XL	P	b, c	c:B	none stated
Translation Tag Team [44]	ASE 2026 Res.	XL	P, E	b, c, i	c:P	subjective judgment
TransLibEval [56]	FSE 2026 Res.	XL	E	b, c, i	c:B	none stated
TRAVEL [33]	ASE 2026 Ind.	XL	P	b, c, e	c:B, e:R	weak tests
UniTrans [58]	FSE 2024 Res.	XL	P, E	a, c	c:B	misleading similarity
VERT [57]	ASE 2025 Res.	XL	P	b, f, g	f:F	incomplete checks
VIM-PT [13]	ASE 2024 Res.	XL	N	a	–	none stated
WCA4Z COBOL-to-Java [8]	ICSE 2026 Ind.	XL+L	P	h	–	none stated
I3 tier: cross-language equivalence detection						
EquivFusion [61]	FSE 2026 Demo	XL	N	g	–	–
Moumoula et al. [39]	FSE 2025 Res.	XL	E	h	–	–